

Overview

The original Bank class acted as a god class because it stored data for several domain objects and also implemented most of their behaviour. The re-engineering keeps the visible behaviour of the client program while moving the detailed operations closer to the objects that own the data.

Account

The account-related behaviour was moved from Bank into Account. The methods for adding funds, applying interest, and closing an account now belong to Account because Account owns the customer reference, balance, and interest rate.

Bank.add_funds now passes only the amount to Account.add_funds. Bank.add_interest delegates to Account.add_interest. Bank.close_account calls Account.close_account, which clears the customer reference and resets the balance to zero, then Bank removes the account from its account list. In this design Bank remains responsible for maintaining the collection of setting up new accounts, while Account is responsible for changing its own financial state.

This follows the reengineering pattern "move behaviour close to data". The balance calculation no longer has to navigate through getters and setters from Bank. If another client or service later needs to deposit money or apply interest, the same behaviour is available directly on the Account object instead of being duplicated elsewhere.

Customer

The customer address and phone number dictionaries were removed from Bank. Previously Bank contained customer_addresses and customer_phone_numbers, which meant customer contact details were stored away from the Customer object itself. This made the model harder to understand because a Customer instance did not clearly contain its own customer data.

The refactoring moved address and phone_number into Customer instances. A new Customer.set_new_customer method initializes a new customer with the default address and phone number values. Bank.setup_new_account still checks whether the customer is already known, but when a new customer is added it initializes that Customer object directly with "NO ADDRESS" and "NO PHONE NUMBER".

This makes each Customer instance responsible for its own contact details. The client assertions were updated carefully so they still verify the same behaviour: a new customer still receives the same default address and phone number, but the checks now read those values from the Customer object rather than from Bank dictionaries.

Branch

The branch opening time was moved into Branch. Previously Bank had a `branch_opening_times` dictionary, so Bank had to know how to store and update information that naturally belongs to each branch. After the refactoring, Branch stores its own `opening_time` and exposes `get_opening_time` and `set_opening_time`.

`Bank.setup_branch` still registers the branch in the bank's branch list, but the default opening time is now set through the Branch object. `Bank.change_opening_time` delegates to `Branch.set_opening_time`. This preserves the same client behaviour: each branch still has an opening time and changing it still works, but the data is now owned by the branch itself.

Staff management was also moved closer to Branch. Branch now provides `add_staff_member` and `remove_staff_member`. `Bank.add_staff_member` delegates to `Branch.add_staff_member`, and `Bank.transfer_staff_member` delegates to `remove_staff_member` on the source branch and `add_staff_member` on the target branch. This removes list manipulation from Bank and prevents repeated navigation code such as `branch.get_staff().append(...)` and `branch.get_staff().remove(...)`. If more clients are added later, they can reuse these branch behaviours rather than duplicating list operations.

Payroll (Navigation Code)

The payroll change was refactored to remove the navigation code pattern. Previously `Bank.change_payroll_date` reached through Payroll into `PaySchedule` and then called `set_pay_date`. This created a chain where Bank needed to know too much about Payroll's internal structure.

A new `Payroll.set_pay_date_by_staff_category` method was added. `Bank.change_payroll_date` now delegates to that method, passing the staff category and date. Payroll remains responsible for finding the correct `PaySchedule` and updating it. `PaySchedule` itself did not need to be changed.

This is an example of removing navigation code. Bank no longer has to know that Payroll stores a `staff_category_pay_schedules` dictionary or that the returned object exposes `set_pay_date`. That knowledge is now contained inside Payroll.

Regression Tests

Some assertions in the client needed to be updated because the location of the data changed. For example, customer address and phone number are now checked through Customer rather than Bank dictionaries, and branch opening time is checked through Branch rather than `Bank.branch_opening_times`. These assertion changes do not remove behaviour; they verify the same externally visible outcomes after responsibility has been moved to the correct domain object. The regression test still confirms that the banking workflow behaves as before.